NexCorpus

Initial Filesystem Documentation

This document defines the initial directory structure for NexCorpus before application files and implementation logic are introduced. The structure is intentionally small and based on current architectural boundaries. Additional directories should be introduced only when a real implementation need appears.

1. Initial Directory Structure

```
NexCorpus/
■
■■■ src/
■ ■■■ app/
■ ■■■ components/
■ ■■■ features/
■ ■■■ lib/
■ ■■■ services/
■ ■■■ types/
■ ■■■ config/
■
■■■ public/
■
■■■ docs/
■ ■■■ architecture/
■ ■■■ decisions/
■ ■■■ security/
■
■■■ scripts/
■
■■■ tests/
■■■ unit/
■■■ integration/
■■■ evaluation/
```

2. Design Principle

Do not create folders simply because they are common in other projects. The initial structure represents known responsibilities. Folders such as hooks, utils, repositories, controllers, adapters, factories, or validators are intentionally not created yet. They can be introduced later if the architecture gives us a concrete reason.

3. Directory Responsibilities

Directory	Responsibility
`src/app/`	Next.js application routes, layouts, pages, route groups, and API entry points. It should not become a dumping ground for business logic.
`src/components/`	Reusable UI components shared across features. The initial ui/ subdirectory can hold generic interface components.
`src/features/`	Feature/domain-oriented application functionality. Expected areas include auth, documents, ingestion, chat, and moderation as those features are implemented.

Directory	Responsibility
`src/lib/`	Low-level infrastructure and integration helpers used by the application, such as database, storage, AI, vector-search, and security integrations when they become necessary.
`src/services/`	Business workflows and orchestration. For example, a document-ingestion service may coordinate validation, moderation, parsing, chunking, embedding, and persistence.
`src/types/`	Shared TypeScript types that need to be reused across application boundaries.
`src/config/`	Application configuration and configuration-related code. Concrete files should be added only when needed.
`public/`	Static assets that are served directly by the Next.js application.
`docs/architecture/`	System and component architecture documentation, including ingestion and retrieval pipeline documentation.
`docs/decisions/`	Architecture Decision Records and other documented technical decisions, including decisions around storage, chunking, and vector search.
`docs/security/`	Security documentation such as threat models, file-ingestion security, and moderation decisions.
`scripts/`	Developer or maintenance scripts that support the project but are not part of the runtime application.
`tests/unit/`	Unit-level tests for isolated application logic.
`tests/integration/`	Tests covering interactions between application components or infrastructure boundaries.
`tests/evaluation/`	RAG-specific evaluation material used to determine whether retrieval is returning the appropriate chunks and whether retrieval quality is improving.

4. Why the Structure Is Feature-Oriented

NexCorpus contains several distinct areas of responsibility: authentication, document management, ingestion, moderation, retrieval, and chat. The features/ directory gives those domains a place to grow without putting all application-specific code into a single components/ or app/ directory.

5. Why lib/ and services/ Are Separate

lib/ represents reusable infrastructure or integration-level capabilities. Examples may include a database client, S3 client, vector-search client, or AI client.

services/ represents application workflows that use those capabilities. For example, an ingestion service may call storage, moderation, parsing, embedding, and persistence components as one business operation.

6. Why tests/evaluation/ Exists From the Beginning

RAG quality cannot be judged only by looking at the final generated answer. NexCorpus will eventually need to evaluate whether the retrieval layer actually returns the relevant chunks. Keeping evaluation as a first-class testing

area prepares the project for retrieval-quality experiments without mixing them with ordinary unit or integration tests.

7. What We Are Intentionally NOT Creating Yet

```
hooks/    utils/    helpers/    constants/
controllers/    repositories/    adapters/    factories/
middleware/    validators/
```

These are not forbidden. They are simply premature at this stage. If a responsibility becomes large enough to justify its own boundary, the directory can be introduced deliberately.

8. Current Project Stage

Project	NexCorpus
Current stage	Initial filesystem and Next.js foundation
Immediate objective	Establish clean directory boundaries before implementation
RAG implementation	Not started
Security implementation	Not started